TEACHING_CODEBASE_GUIDE

Pintop | 2026 Training

Backend Design & Engineering - Teaching Codebase and Lesson Map

PINTOP TECHNOLOGIES LIMITED | 2026 Production Curriculum

This is the companion codebase for the student and instructor materials. Every original programme and live-class filename has been retained; the 108 live classes now include linked code-along exercises.

Language, runtime and setup

Technology: PHP / Laravel 13. The Laravel backend replaces previous Node/Express server examples. JavaScript/React browser clients, Flutter apps and Python security labs remain in their appropriate languages.

```
cd "Backend Design & Engineering/codebase"
composer install
cp .env.example .env
php artisan key:generate
php artisan migrate --seed
php artisan serve --port=8000
```

Check using the programme tools:

```
php artisan route:list --path=api
php artisan test
```

Do not assume an install succeeded. Confirm required dependencies and network availability on the instructors machine. The packed Laravel sources do not include Composer vendor dependencies. The class excerpts are selected sections of real project files, not promised standalone executables.

Architecture and where to find the teaching work

- `codebase/` holds the supplied application source or editable programme templates.
- `lesson-labs/Class NN — Topic/REFERENCE EXCERPT.*` contains a copy of source lines from the real programme file.
- `lesson-labs/Class NN — Topic/01_CODE_ALONG_LAB.md` contains the beginner typing exercise, checks and assessment.
- `lesson-labs/Class NN — Topic/02_INSTRUCTOR_ANSWER_KEY.md` provides instructor prompts and expected evidence.
- The corresponding live-class guide in `2026 Production Curriculum/live-classes/` tells the instructor precisely when to read, type, test and question students.

Course-to-code map

Class	Topic	Real project reference	Lab folder
01	REST Principles & Resource Modelling	`routes/api.php`	`lesson-labs/Class 01 — REST Principles & Resource Modelling`

Class	Topic	Real project reference	Lab folder
02	HTTP Semantics, Versioning & Pagination	`routes/api.php`	`lesson-labs/Class 02 — HTTP Semantics, Versioning & Pagination`
03	Routing, Middleware & Request Flow	`bootstrap/app.php`	`lesson-labs/Class 03 — Routing, Middleware & Request Flow`
04	Validation, Errors & Logging	`app/Http/Controllers/PayTrackController.php`	`lesson-labs/Class 04 — Validation, Errors & Logging`
05	Relational Modelling & Normalisation	`database/migrations/2026_01_01_000002_paytrack.php`	`lesson-labs/Class 05 — Relational Modelling & Normalisation`
06	Migrations, Transactions & Query Optimisation	`database/migrations/2026_01_01_000002_paytrack.php`	`lesson-labs/Class 06 — Migrations, Transactions & Query Optimisation`
07	Password Handling & Token-Session Design	`app/Http/Controllers/PayTrackAuthController.php`	`lesson-labs/Class 07 — Password Handling & Token-Session Design`
08	Authorisation, RBAC & API Security	`app/Http/Middleware/BearerToken.php`	`lesson-labs/Class 08 — Authorisation, RBAC & API Security`
09	Redis, Cache Design & Invalidation	`config/cache.php`	`lesson-labs/Class 09 — Redis, Cache Design & Invalidation`
10	Rate Limiting & Performance Measurement	`routes/api.php`	`lesson-labs/Class 10 — Rate Limiting & Performance Measurement`
11	Unit-Integration Testing & OpenAPI	`tests/Feature/PayTrackTest.php`	`lesson-labs/Class 11 — Unit-Integration Testing & OpenAPI`
12	Deployment, Health Checks & Capstone Review	`routes/api.php`	`lesson-labs/Class 12 — Deployment, Health Checks & Capstone Review`

Troubleshooting before class

1. **Wrong working directory** - read the path in the course map; server and client may have different roots.
2. **Missing tooling** - confirm versions, run the relevant package installer and resolve OS permission errors; do not try to execute a partial source excerpt.
3. **API not reachable** - confirm that the Laravel server starts locally and that the client URL points at the right port; an Android emulator uses `10.0.2.2` for the host, not `localhost`.
4. **Failed migration** - use disposable seeded teaching data and confirm `.env` connection; never reset a real database.
5. **Failing test** - inspect the first assertion failure and ensure fixtures or the test database are correct; record both the failing and repaired output.

6. **Security exercises** - use the bundled local classroom application or another explicitly authorised target only.

Instructor delivery rules

Use exactly 120 minutes per class according to its read-aloud guide. Students should see real code typed in stages, see a run or verification and be able to explain a failure. Show real outputs on your machine; do not say a command passed unless it actually passed.